

FundedNext — Product Analyst (Growth)

The brief

There are **two parts**. Both go into a single memo.

Part 1 — Acquisition

FundedNext has **\$200,000** available next quarter for a customer acquisition campaign. We want your recommendation:

1. **Which trader segment should we target with the \$200K, and why?**
2. **What return should we expect on that spend?** (Be explicit about how you define “return” and what assumptions sit behind your number.)
3. **How should we measure whether the campaign worked?** Design the experiment / measurement plan.

Part 2 — Pricing & promotions

FundedNext runs frequent **sitewide discount campaigns**, and we are debating whether to lean *harder* into discounting next year, and on which plans. Using the discount information in `transactions.parquet` :

1. **Estimate how price-sensitive each plan is.** Which plans respond to a discount, and which don't?
2. **Recommend which plan(s) to discount and which to leave at full price** — with your reasoning.
3. **Quantify what a deeper discount on your recommended plan would be worth** — net of cannibalisation (buyers who would have purchased anyway) and any downstream effects you can find in the data.

The data you've been given is what FundedNext actually has — five files, extracted from different systems, with the inconsistencies you would expect from any real production environment. **You will need to clean and reconcile them before you can analyse them.** Use any tools you like.

What's in the box

This folder contains 5 files plus this README. **No PII is present** — all identifiers are synthetic codes. The data is a **realistic snapshot taken on 2026-05-15** of FundedNext's CFD + Futures business.

File	Format	Rough size	What it is
<code>plans.csv</code>	CSV	10 rows	Plan catalogue (product dimension)
<code>accounts.csv</code>	CSV	~50K rows	One row per trading account (the dimension)
<code>transactions.parquet</code>	Parquet	~114K rows	Account purchases + resets (revenue events)
<code>payouts.jsonl</code>	JSON Lines	~8K rows	Trader payout records
<code>lifecycle.csv</code>	CSV	~42K rows	CFD-only per-account journey rollup

Domain primer (so you don't have to Google FundedNext)

FundedNext is a **proprietary trading firm**. The business model is:

1. A retail trader **buys a “Challenge”** — a simulated trading account (`plans.csv`). Common variants: **Stellar 1-Step** (one challenge phase), **Stellar 2-Step** (two phases), **Stellar Lite** (cheaper, two phases), **Stellar Instant** (skip the challenge, get funded immediately), and **Futures Real**.
2. They trade on a demo environment. If they hit profit targets without blowing the account, they **pass to the next phase**. If they pass all phases, they become **funded** (`phase = 'Real'`).
3. Once **funded**, they trade with firm capital. Profits they make get paid out to them as **payouts**.
4. If they break risk rules (max daily loss, max overall loss, inactivity), the account is **breached** and closes. Many traders then **reset** the same account (small fee, fresh start) or **buy a new account**.
5. The firm's **revenue** is **challenge purchases + reset fees**. Its **cost of goods sold (COGS)** is trader **payouts + platform fees** (the per-account fee FundedNext pays the trading-platform provider).

So **net contribution to the firm** = revenue – COGS. The \$200K must be deployed where the **marginal acquired customer is most net-positive** to the firm over a reasonable observation window.

Glossary

- **Phase:** `P1` , `P2` , `Real` (CFD) or `Demo` , `Real` (Futures). `Real` = funded.
- **Status:** `funded` (active), `breached` (closed via rule-break), `running` (still in challenge), `inactive` (idle).
- **Reset:** same account, fresh attempt, lower price.
- **Repurchase:** same customer buys a **new** account → new `account_id` .
- A **customer** can hold multiple accounts. A **journey** is one account's lifecycle from P1 to its end-state.

File schemas

We're giving you column names — **not types, not data-quality notes**. Treat them as you'd treat any real production export: validate yourself.

plans.csv

```
plan_id, plan_name, plan_type, price_usd, market
```

accounts.csv

```
account_id, customer_code, plan_id, country_code, server_type,  
created_at, updated_at, deleted_at, phase, status, breached_by
```

transactions.parquet

```
transaction_id, account_id, customer_code, plan_id,  
amount, total, discount_pct, payment_method, transaction_type, created_at_utc
```

`transaction_type` values: `purchase`, `purchase_refunded`, `reset`. `discount_pct` is the discount applied to that transaction (0 = full price).

payouts.jsonl

```
{"payout_id": "...", "account_id": ..., "customer_code": "...",  
  "plan_id": ..., "requested_at": "...", "disbursed_at": "...",  
  "real_disbursement_amount": ..., "commission_amount": ...,  
  "total_disbursed_amount": ..., "payout_category": "..."}  
...
```

`payout_category` values: `Performance`, `Brand Promise`.

lifecycle.csv (CFD only)

```
journey_id, customer_code, plan_id,  
p1_login, p1_status, p2_login, p2_status, real_login, real_status,  
subscription_revenue, reset_revenue, reset_count,  
lifetime_total_payout, first_purchase_date
```

What to send back

Three deliverables, in a single zip or shared folder:

1. **Memo** (`memo.pdf` or `memo.md`) — **3 pages maximum** (roughly 2 for Part 1, 1 for Part 2). Lead each part with your recommendation. Show your numbers, your assumptions, and how you'd measure success. Anything that doesn't directly support a recommendation goes into the appendix or stays in your code.
2. **Headline charts** (`chart*.png` / `.pdf`) — up to **two** visuals: one for the acquisition decision, one for the pricing decision.
3. **Code** (`notebook.ipynb` or `analysis.py`) — must run end-to-end against the five files in this folder. Include any `requirements.txt` or environment notes.

You may add a brief technical appendix (`appendix.md` , optional, ≤ 1 page) for caveats, validation checks, or things you didn't have time for.